Osvrt na Zadatak 1 — osnovni DPLL

Bilješke prije nego krenem na proširenja iz Zadatka 2.

Što je napravljeno

Osnovni DPLL po 9. predavanju s tri sastavnice:

- **PJK** (Unit Clause Propagation) — naivna varijanta: u petlji prolazim kroz sve klauzule, brojim nepridijeljene literale po klauzuli; ako ih je točno 1 i nijedan nije `true`, forsiram tu vrijednost. Loop dok se neka klauzula promijeni.
- **UKČL** (Pure Literal Elimination) — za svaku nezadovoljenu klauzulu pamtim u kojim polaritetima se varijabla pojavljuje; pridijelim onu polarnost koja čini klauzulu istinitom ako je samo jedna od polariteta prisutna.
- **DLIS** za grananje — biram (varijabla, polaritet) s najvećim brojem pojavljivanja u trenutno nezadovoljenim klauzulama. To nije slučajna heuristika (specifikacija to zabranjuje), a poštedi nas kasnijih grananja jer maksimalno reducira formulu.

Kronološki povratak je riješen preko *traila* (stog dodjela) — `assignLit` push, `backtrackTo(mark)` pop. Bez kopiranja formule, vrlo brzo.

Što sam naučio iz mjerenja

Iz tablice u `README.md`:

Problem	Veličina	Vrijeme
Mali / strukturirani SAT	<200 klauzula	< 15 ms
`medium.cnf` (Blocks World)	953 klauzula	9 ms
`flat50-1.cnf` (3-coloring)	545 klauzula	11 ms
`hole7.cnf` (pigeonhole UNSAT)	204 klauzule	61 ms (5039 odluka)
`hanoi4.cnf` (SAT)	4934 klauzule	TIMEOUT (>5 min, 1.34M odluka)
`bf0432-007.cnf` (UNSAT)	3668 klauzula	TIMEOUT (>60 s)

Zaključci:

1. **DLIS dobro razbija male probleme** — `medium.cnf` u 12 odluka i 9 ms je praktički "samo" PJK + dobar prvi pogodak. Klasični "free meal".
2. **Pigeonhole je tipičan worst-case za bilo koji DPLL bez CDCL**, ali pri ovako maloj veličini ipak ga DLIS-vođeni osnovni DPLL zaključi za ~60 ms. Za `hole8`+ bi se to već pokazalo kao eksponencijalna eksplozija.
3. **`hanoi4` i `bf0432` ne idu** — i to je upravo ono što motivira Zadatak 2. Iz statistike vidimo zašto: `hanoi4` napravi **1.34 milijuna grananja** i **95.7 milijuna unit-propagacija** u 5 minuta. Sve je to "rad bez učenja" — svaki put kad bismo pogriješili visoko u stablu odluka, ponovo upadamo u istu konfliktnu podgranu jer si ne pamtimo zašto je ona loša.

Uska grla osnovnog DPLL-a (motivacija za Zadatak 2)

- **Nema učenja iz konflikata.** Konflikt nam samo kaže "vrati se i probaj drugačije", ali ne razumijemo *zašto* je došlo do konflikta. Posljedica: ponovno istraživanje istih konfliktnih grana s drugačijim "okolnostima". → rješenje: **CDCL + 1UIP klauzule**.
- **Kronološki povratak je previše konzervativan.** Konflikt na 6. razini može biti uzrokovan odlukom na 2. razini, a osnovni DPLL će ipak proći kroz 3., 4. i 5. razinu. → rješenje: **nekronološki povratak (backjumping)** preko grafa implikacija.
- **PJK kakva ja imam je naivna O(klauzula × literala) po prolazu.** Za 4934 klauzula u `hanoi4` to je smiješno skupo, jer tijekom propagacije klauzule baš i ne mijenjaju često svoj "kritični" status. → rješenje: **two-watched literals** (Chaff) — u svakoj klauzuli pratimo samo dva literala, ostale ignoriramo.
- **Pretraga zna ostati u "lošem dijelu" prostora.** Restart napušta trenutno stanje pretrage uz zadržavanje naučenih klauzula i kreće ispočetka, čime se može pogoditi puno bolji put. → rješenje: **restart s Luby nizom**.

Plan za Zadatak 2

Implementirati u zasebnoj klasi `CdclSolver.java` (osnovni `DpllSolver` ostaje netaknut radi usporedbe) sljedeća **četiri značajna proširenja**:

1. **CDCL — 1UIP učenje klauzula** (značajno)
2. **Graf implikacija + nekronološki povratak (backjumping)** (značajno)
3. **Two-watched literals** za PBO (značajno; Chaff)
4. **Restart s Luby nizom** + zadržavanje naučenih klauzula (značajno)

Plus uz CDCL ide **VSIDS** heuristika (ne računa se kao zasebno proširenje prema specifikaciji, ali je standardna pratilja CDCL-a).

Parametrizacija (zahtjev specifikacije: *"program treba omogućiti parametrizaciju toga koja proširenja koristi, a koja ne"*):

```
java Main --solver=cdcl ulaz.cnf                  # sva proširenja
java Main --solver=cdcl --no-restart ulaz.cnf
java Main --solver=cdcl --no-backjump ulaz.cnf    # CDCL bez backjumpa
java Main --solver=dpll ulaz.cnf                  # osnovni DPLL iz Z1
```